

用 TypeScript 與 AI 打造機器人

第一部：機構篇 — 機器人基礎與 TypeScript 程式設計

建議時長：約 3 小時（含休息）

一、本部分定位

本課程分為兩部分進行。第一部（本文件）聚焦於「機構」層面：認識 Robot Inventor 硬體、建立 TypeScript 開發環境，並透過馬達與感測器的基本操作，理解機器人「感測 → 判斷 → 動作」的控制迴路。學生在本部分結束時，應能獨立寫出規則邏輯（if / else）驅動的反應式機器人行為。

第二部將在此基礎上，把規則邏輯替換為 AI 決策，並加入人機協作除錯與總結專題。兩部分建議間隔至少一天，讓學生消化本部分內容後再進入 AI 章節，避免概念一次疊加過多。

二、學習目標

- 能說明 Hub、馬達、感測器在控制迴路中的角色
- 能安裝並設定 Node.js、TypeScript、node-poweredup 開發環境
- 能撰寫 TypeScript 程式連接 Hub、控制馬達
- 能使用事件監聽（event listener）持續讀取感測器資料
- 能結合馬達與感測器，寫出至少一個條件式（規則邏輯）反應行為

三、適用對象與先備知識

- 國高中或大學階段學生，具備基礎程式邏輯概念（變數、條件式、迴圈）即可
- 不需要事先具備 TypeScript 或 Python 經驗
- 建議 2 人一組進行，共用一台 Robot Inventor 套件

四、所需教具與軟體

LEGO Mindstorms Robot Inventor 套件	含 Hub、馬達、感測器（顏色 / 距離）
電腦（Windows / macOS / Linux）	需支援藍牙（Bluetooth Low Energy）
Node.js（v18 以上）	JavaScript / TypeScript 執行環境
TypeScript、ts-node	撰寫並執行 TypeScript 程式
node-poweredup（npm 套件）	官方相容 Robot Inventor 的 Node.js SDK

五、時間表

1	環境建置與硬體介紹	45 分鐘	0:45

2	機器人基礎概念	20 分鐘	1:05
3	第一支 TypeScript 程式	35 分鐘	1:40
休息	中場休息	10 分鐘	1:50
4	讀取感測器資料	35 分鐘	2:25
5	撰寫反應式行為（規則邏輯）	45 分鐘	3:10

💡 階段 1 的時間已從原先 30 分鐘上修至 45 分鐘 —— 藍牙配對是最常拖延進度的環節，多台學生電腦同時配對容易出現訊號干擾或驅動問題，預留緩衝時間可避免後續階段被壓縮。

六、逐步教學內容

階段 1：環境建置與硬體介紹（45 分鐘）

教學目標：完成開發環境安裝，並認識 Robot Inventor Hub 的實體元件。

教學活動：

- 安裝 Node.js（v18 以上）與 TypeScript
- 執行 `npm install node-poweredup` 安裝 SDK
- 以藍牙配對 Robot Inventor Hub 與電腦
- 在實體套件上指認 Hub、馬達接口、感測器接口

```
# 建立專案並安裝套件
mkdir robot-lesson && cd robot-lesson
npm init -y
npm install typescript ts-node node-poweredup --save
npx tsc --init
```

💡 常見狀況排除：若 Hub 掃描不到，先確認 Hub 已開機（燈號亮起）、藍牙未被其他裝置佔用連線；Linux / 樹莓派需額外設定藍牙裝置存取權限，建議教師事先在自己的機器上測試一次完整流程。

階段 2：機器人基礎概念（20 分鐘）

教學目標：理解機器人「輸入 → 處理 → 輸出」的基本迴路，此階段不涉及程式碼，適合作為動手前的討論暖身。

- 介紹 Hub（大腦 / 處理）、馬達（輸出）、感測器（輸入）的角色
- 讓學生在實體套件上指認各連接埠（Port A、B、C...）
- 討論：真實世界中還有哪些系統符合這個迴路？（例如恆溫器、自動門、冷氣機）
- 延伸提問：如果拿掉感測器，機器人還能『反應』環境嗎？

階段 3：第一支 TypeScript 程式（35 分鐘）

教學目標：驗證「TypeScript → 硬體」的完整流程可以順利執行，建立成就感，同時讓學生熟

悉 npx ts-node 的執行方式與常見編譯錯誤訊息。

```
import { PoweredUP, Hub } from "node-poweredup";

const poweredUP = new PoweredUP();

poweredUP.on("hub", async (hub: Hub) => {
  console.log(`已連接 Hub : ${hub.name}`);
  await hub.connect();
  await hub.setMotorSpeed("A", 50); // A 埠馬達以 50% 動力運轉
  await new Promise((r) => setTimeout(r, 2000));
  await hub.setMotorSpeed("A", 0); // 停止
  await hub.disconnect();
});

console.log("正在搜尋 Hub...");
poweredUP.scan();
```

執行方式：npx ts-node motor.ts，觀察馬達是否依指令啟動與停止。

延伸練習（若時間允許）：讓學生嘗試調整動力數值（power）與運轉時間，觀察馬達行為的變化，並改為控制不同連接埠的馬達。

階段 4：讀取感測器資料（35 分鐘）

教學目標：從「單次下指令」進階到「持續接收回饋」的概念，理解事件監聽（event listener）與一次性指令的差異。

```
poweredUP.on("hub", async (hub: Hub) => {
  await hub.connect();
  hub.on("colorAndDistance", (port, data) => {
    console.log(`顏色：${data.color}，距離：${data.distance} cm`);
  });
});
```

討論重點：事件監聽（on）與一次性指令有何不同？為什麼機器人需要「持續」感知環境，而不是只問一次？

💡 此階段的感測數值可能會跳動或不穩定（尤其距離感測器），這是正常的硬體雜訊現象，可藉此機會與學生討論「感測器雜訊」與「濾波」的概念，為第二部的 AI 決策鋪陳——AI 同樣需要處理不完美、有雜訊的輸入。

階段 5：撰寫反應式行為（規則邏輯）（45 分鐘）

教學目標：結合馬達與感測器，寫出第一個「條件式行為」，鞏固控制迴路概念。此階段刻意不使用 AI，目的是讓學生清楚體會「規則邏輯」的樣貌，作為第二部與 AI 決策對照的基準。

```
hub.on("colorAndDistance", async (port, data) => {
  if (data.distance < 10) {
    await hub.setMotorSpeed("A", 0); // 太近，停止
  } else {
    await hub.setMotorSpeed("A", 40); // 繼續前進
  }
});
```

```
}  
});
```

延伸練習：改成沿黑線行走的邏輯，或加入轉彎判斷（例如偵測到障礙物時先後退再轉向）。

第一部總結提問（可留給學生課後思考，銜接第二部）：目前這段程式的規則都是『人類事先寫死』的。如果環境比原本設想的更複雜（例如障礙物形狀不規則、光線變化影響顏色感測），這種寫死的規則還夠用嗎？

七、第一部評量重點

- 能否獨立完成藍牙配對與環境安裝（不依賴教師逐步指導）
- 程式能否正確控制馬達啟動與停止
- 是否理解事件監聽與一次性指令的差異，並能用自已的話說明
- 規則邏輯練習是否能正確反映『感測 → 判斷 → 動作』的迴路

八、教師提醒

- 藍牙連線偶有不穩定情形，建議每組保留 5-10 分鐘的重新配對緩衝時間
- 若學生對 TypeScript 語法感到陌生，可在階段 3 前額外安排 10-15 分鐘的語法暖身（變數型別、箭頭函式、async/await 的基本概念）
- 建議在階段 5 結束時，保留每組簡短分享一分鐘，讓全班看到不同的規則邏輯寫法，為第二部『規則 vs. AI 決策』的對比討論鋪墊